In-Depth Analysis of Homomorphic Encryption Libraries

Itai Zloclzower Jacob Shemesh Tomer Ovadya Yoni Glickstein

Ben-Gurion University of the Negev

Abstract

Homomorphic encryption lets a server compute on data it cannot read. Its cost is often summarised using isolated operations under unmatched configurations; we show that both choices substantially affect the result. Re-running OpenFHE at TenSEAL's ring dimension and scaling factor cuts its multiplication penalty from 6.89× to 3.16× and its geometric mean over six operations from 6.44× to 3.05×, substantially narrowing the measured gap between the libraries while a 3.05× difference remains. A complete 1000-patient healthcare workload runs at 1,598× plaintext against the 13,500× its constituent operations predict in isolation — 214 ms end to end, matching the plaintext answer to five significant figures — with ciphertext expansion falling from 8,361× to 132×: nothing became faster, but CKKS charges per ciphertext, so 1024 filled slots amortise what 5 do not. Within one library the per-operation overhead still spans 153× for addition to 33,000× for summation, set by whether the operation moves data between slots. Every timing is a mean over repeated measurements behind one shared harness, reported with a correlation-aware confidence interval, and every encrypted result is checked against its plaintext value before its timing is reported.

1 Introduction

Encryption protects data at rest and in transit, but not while it is being used: to compute on encrypted data a normal system decrypts it first, exposing the plaintext to whatever machine does the work. Homomorphic encryption (HE) removes that step — operations on ciphertexts produce ciphertexts that decrypt to the correct answer, so a cloud provider can process data it never sees.

Mature open-source implementations now make that capability usable in practice, which turns it into a concrete engineering decision: if a hospital or a bank wanted to outsource a computation over sensitive records today, what would it cost, and which library should it choose? Answering that needs measurement. Published complexity results say little about what a particular library does with a particular operation on a particular machine.

This project supplies that measurement. We compare two CKKS libraries, **TenSEAL** and **OpenFHE**, against an unencrypted baseline over a set of operations common in data analysis, and report runtime, memory, ciphertext size, accuracy, and the practical problems we met while building them.

Research question. *What is the practical overhead of performing basic computations using two different homomorphic encryption implementations, compared to equivalent plaintext computation?*

What this report establishes. We implement five comparable computations in plaintext, TenSEAL and OpenFHE, and additionally measure the HE-specific key generation, encryption and decryption, all behind a single measurement harness, so the three columns of every table differ only in the call being timed. Beyond the per-operation costs that setup yields three results. Re-running OpenFHE at TenSEAL's ring dimension and scaling factor attributes most of the measured gap between the libraries to their unmatched configurations rather than to the libraries themselves. Whether an operation must move data between ciphertext slots, rather than the amount of data it processes, sets its cost: addition runs at 153× plaintext and summation at

33,000×. And a complete 1000-patient workload runs at 1,598× plaintext, an eighth of what the isolated operations predict.

The report follows the measurements in the order they led to one another. Section 2 explains the scheme and the libraries, Section 3 what we built and Section 4 how we measured it. Section 5 gives the cost of each operation; Section 6 asks whether that measures the libraries or the settings we gave them. Section 7 combines everything in a healthcare example, and Sections 8–9 cover usability and limitations.

2 Background

2.1 Homomorphic encryption and CKKS

Both libraries implement **CKKS** [1], a scheme for approximate arithmetic on real numbers. Approximate is the important word: decrypting a CKKS result gives the correct value plus a small error, in exchange for much better performance than exact schemes on real-valued data. For statistics over physical measurements of the kind considered here, the error we observed stayed well below the precision of the input, as Section 5.5 reports.

Four properties of CKKS shape every measurement in this report.

- **Ring dimension.** A ciphertext is a pair of polynomials of degree below N, the ring dimension, and N drives the cost of every operation. Security is set by the ring dimension and the ciphertext modulus together, not by either alone: for a fixed modulus, a bigger ring raises the security level and slows every operation, though not by the same factor for each — Section 6 measures slowdowns from 2.1× to 7.8× across operations for one such change.
- **Slot packing.** One ciphertext holds many values in parallel "slots", and one operation acts on all of them at once. For addition and multiplication the cost depends on the ring dimension, not on how many slots hold real data; operations that aggregate across slots are the exception, since their rotation count grows with the number of slots involved.
- **Multiplicative depth.** Each multiplication uses up one level from a budget fixed at setup. Exceeding it is a correctness problem rather than a performance one: in our runs both libraries raised an error rather than returning a degraded result, and the parameters had to be enlarged before the computation would run at all.
- **Rotations.** Slots cannot be read individually. Summing across them means rotating the ciphertext and adding, about $\log_2(\text{slots})$ times, and every rotation needs an expensive key switch. This is why summation costs so much more than addition.

2.2 The two libraries

TenSEAL [3] is a Python library built on Microsoft SEAL. It presents encrypted vectors as ordinary Python objects that support `+`, `*` and `.sum()`, and handles relinearization and rescaling internally. The programmer gives it the modulus chain directly.

OpenFHE [2] is a C++ library with Python bindings, maintained by an open-source community of academic and industry contributors. It exposes lower-level calls (`EvalAdd`, `EvalMult`, `EvalSum`) and needs evaluation keys to be generated explicitly. The programmer gives it a multiplicative depth and a scaling size, and the library works out the ring dimension that satisfies them at the requested security level.

That difference in interface has a measurable consequence. TenSEAL has no default CKKS configuration to adopt — the modulus chain and the global scale must both be supplied, and we used the example configuration from its own documentation. OpenFHE does ship defaults, and from the depth and 128-bit security level we requested it derived ring dimension 16384 against TenSEAL's 8192. The two arms therefore ran at different ring dimensions, which Section 6 quantifies.

3 Implementation

We implement five comparable computations in plaintext, TenSEAL and OpenFHE, and additionally measure the HE-specific operations that have no plaintext counterpart: key generation, encryption and decryption.

Operation	What it does
Key generation	Build the crypto context and generate the keys the library needs
Encryption	Encode a vector of numbers into a ciphertext
Addition	Element-wise sum of two encrypted vectors
Multiplication	Element-wise product of two encrypted vectors
Summation	Sum of all elements inside one encrypted vector
Average	Summation, then multiplication by $1/n$
Dot product	Element-wise multiplication, then summation
Decryption	Recover the plaintext result

Table 1: The implemented operations. The five arithmetic operations have plaintext counterparts, which form the baseline; key generation, encryption and decryption are HE-specific.

The two libraries need different code for the same computation. Encrypted addition in TenSEAL is `enc + enc`; in OpenFHE it is `cc.EvalAdd(ct, ct)`. Summation is `enc.sum()` against `cc.EvalSum(ct, n)`, and the OpenFHE version needs summation keys generated beforehand.

One incompatibility showed up immediately and affected everything after it: **OpenFHE requires the batch size to be a power of two**, while TenSEAL accepts any length. Every input has to be padded up to the next power of two. For simple operations that is harmless, but Section 7 shows a case where the padding *value* has to be chosen carefully.

3.1 The three experiments

	Experiment	Question it answers
1	Basic operations (§5)	What does each encrypted operation cost, against the same operation in plaintext?
2	Matched parameters (§6)	Is the gap between the libraries, or between their initial unmatched configurations?
3	Healthcare use case (§7)	What happens when the operations are combined into one complete workload?

Table 2: The three experiments and the question each one answers.

4 Experimental Setup

4.1 What we measure, and how

Runtime. Each operation is timed with `time.perf_counter` around a single call. Encryption and decryption are timed separately, so the cost of getting data in and out is never folded into the cost of computing on it.

Repetitions and warm-up. One timing of an HE operation is not reproducible. The primitive-operation figures in Section 5 are each the **mean of 1000 timed repetitions** after **50 warm-up calls that are discarded**; the healthcare pipeline uses 1000 and 20, the matched-parameter experiment 200 and 20, and key generation 50 and 5. The first calls into either library pay one-off costs — loading the native library, building lookup tables, faulting in fresh memory — that would inflate the mean and make it depend on the repetition count. Garbage collection is off inside the timed loop, and every repetition starts from freshly encrypted inputs, so none inherits noise or a consumed level from the one before it.

Memory. Two figures. `tracemalloc` sees only Python-level allocations, and both libraries keep ciphertext coefficients in native C++ memory, so we also allocate 50 ciphertexts, hold them,

and measure the growth in process resident set. Memory is measured in a separate pass from timing, because `tracemalloc` hooks every allocation and distorts short measurements.

Uncertainty. For every operation the harness records the standard deviation, median, trimmed mean, interquartile range, 95th and 99th percentiles and a 95% confidence interval on the mean, and stores all 1000 samples. It also splits the samples in half and compares the two means, flagging any measurement whose halves disagree by more than 5%, which catches a machine that changed underneath the benchmark — a failure a confidence interval alone does not detect. Section 5.2 reports these figures.

Ciphertext size. Each library's own serialization routine, in bytes, compared against the plaintext vector stored as 8-byte doubles.

Accuracy. Every encrypted result is compared against the plaintext computation of the same values, and we report the largest absolute error. Each benchmark validates this error against a correctness threshold before reporting any timing, so a misconfigured run fails visibly.

4.2 Machine and configuration

Measurements ran on a compute node reserved exclusively for the job, so nothing else shared the machine. We fixed the thread count to one (`OMP_NUM_THREADS=1`): both libraries use multithreaded native code, and on a shared machine their measured cost would depend on how many cores each happened to get, which is not a property of the library.

CPU	Intel Xeon E5-2680 v3 @ 2.50 GHz, single-threaded, exclusive node
OS / Python	Rocky Linux 9.7 / Python 3.8.20
Libraries	TenSEAL 0.3.15, OpenFHE 1.4.1, NumPy 1.24.4
TenSEAL parameters	`poly_modulus_degree` = 8192, coefficient moduli $[60, 40, 40, 60]$, scale 2^{40}
OpenFHE parameters	multiplicative depth 3, scaling modulus 50 bits, 128-bit security ⇒ ring 16384
Input	The vector $[1.0, 2.0, 3.0, 4.0, 5.0]$ unless stated otherwise

Table 3: Initial configurations used in the basic-operation benchmark. They are not equivalent to each other; Section 6 examines the effect of matching selected parameters.

Every timed measurement retains its full sample set, so each table can be inspected without re-running the benchmark. Reproduction notebooks and the complete sample data are included in the accompanying submission archive.

5 Results: The Cost of Basic Operations

5.1 Runtime

Operation	**Plaintext (ms)**	**TenSEAL (ms)**	**OpenFHE (ms)**	**TenSEAL overhead**	**OpenFHE overhead**
Encrypt	—	5.4914	15.1101	—	—
Add	0.000629	0.0961	0.3869	153×	615×
Multiply	0.000550	4.7413	32.4827	8,616×	59,025×
Sum	0.000320	10.5082	76.1546	32,832×	237,943×
Average	0.000349	11.7961	81.3563	33,794×	233,072×
Dot product	0.000993	11.0764	96.6155	11,152×	97,272×
Decrypt	—	1.4029	22.6702	—	—
Total (5 comparable ops)	**0.002841**	**38.2180**	**286.9961**	**13,450×**	**101,003×**

Table 4: Mean runtime over 1000 repetitions. The total covers the five operations that exist in all three columns; encryption and decryption have no plaintext counterpart and are left out of it. Ratios are approximate: the plaintext column sits near the timer floor, as Section 5.2 shows.

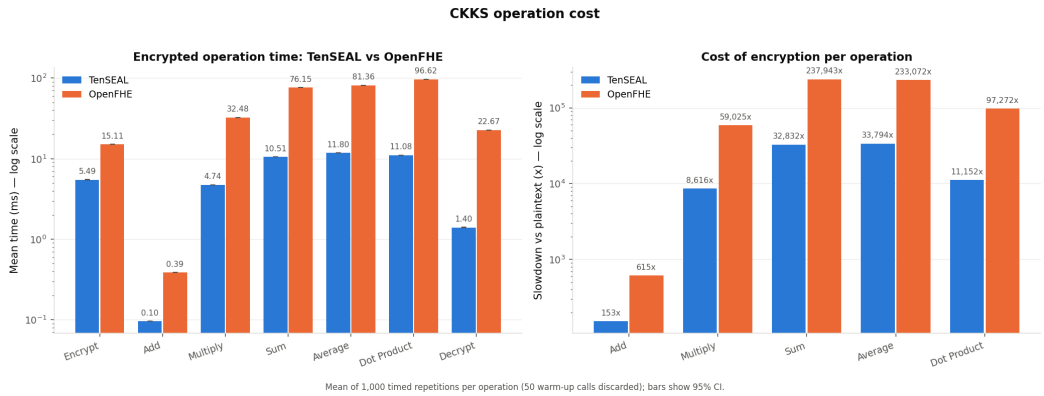

Figure 1: Runtime per operation on a logarithmic scale, and the slowdown against plaintext. Both libraries span two orders of magnitude across operations.

There is no single overhead figure. Inside TenSEAL alone the cost runs from 153× for addition to 33,000× for summation. A single quoted figure such as "HE is ten thousand times slower" is therefore of limited use, since the answer depends on which operation dominates the workload.

Addition was the least expensive encrypted operation measured. Encrypted addition takes 96 microseconds, a fiftieth of a multiplication and a hundredth of a summation. The reason is structural. Addition works on ciphertext coefficients directly. Summation, average and dot product have to move data between slots, and every rotation carries a key switch. This one mechanism explains the order of every row in the table and gives the practical rule that follows from it: a computation built from additions costs one to two orders of magnitude less than one that aggregates across slots.

TenSEAL is faster here, on every operation. It leads by 2.75× on encryption and 6.85× on multiplication. Section 6 tests whether that reflects the libraries or the parameters we gave them.

5.2 How precise are these numbers?

The mean must be read together with its sampling variability, and the two sides of the comparison are not measured equally well.

Operation	Plaintext	TenSEAL	OpenFHE
Encrypt	—	0.82	0.13
Add	8.80	0.26	0.11
Multiply	0.26	1.32	0.10
Sum	3.12	0.12	0.11
Average	0.15	1.33	0.38
Dot product	4.77	0.13	0.07
Decrypt	—	0.80	0.23

Table 5: Half-width of the 95% confidence interval on the mean, as a percentage of that mean, computed by non-overlapping batch means over 25 batches. One measurement was flagged for drift between the first and second half of its samples: the plaintext dot product, at −5.1 %. No encrypted measurement was flagged.

These intervals are computed from batch means rather than by treating the 1000 samples as independent. That distinction matters here: consecutive timings are correlated — lag-one autocorrelation reaches 0.95 for TenSEAL's average and 0.90 for its multiplication — and an interval assuming independence would report 0.22 % for both, roughly six times narrower than the 1.33 % and 1.32 % above. The narrower figure would be an artefact of the assumption, not a property of the measurement.

Read that way the encrypted measurements are still tight: every encrypted interval is within 1.4 % of its mean, so the ratios in Table 4 are not artefacts of noise. The plaintext column is the weak one. Encrypted addition takes 96 µs, but a plaintext addition takes 0.63 µs, and at that scale the per-call overhead of `time.perf_counter` and the surrounding Python loop is no longer negligible against the operation being timed. The three plaintext operations with the widest intervals — addition, summation and dot product — have relative standard deviations of 110 %, 51 % and 54 %, against at most 4 % everywhere else, and their medians sit below their means, which is the signature of a floor plus occasional interruptions rather than a symmetric distribution.

This matters in one direction only. Timer and loop overhead inflate the plaintext figure, which is the denominator of every ratio, so it underestimates the overhead ratio. The ratios in this report are conservative on that account, and the correct fix — timing plaintext operations in batches and dividing, rather than wrapping each call — would raise them. We did not re-run the suite to do this, so the plaintext column should be read as accurate to roughly a few percent rather than to its final digit.

5.3 Key generation

Key generation happens once per session rather than once per operation, so we timed it separately, over 50 repetitions.

	TenSEAL	**OpenFHE**
Context and key generation	216.180 ms	208.308 ms

This is the one measurement where OpenFHE leads, and the two are within 4% of each other. The scale is what matters: TenSEAL spends 216 ms building keys against 38 ms for all five encrypted operations, so a service building a fresh context per request would spend most of its time on key generation.

5.4 Memory and ciphertext size

Measure	**TenSEAL**	**OpenFHE**
Python-visible memory (`tracemalloc`), encryption	0.59 KB	0.42 KB
Resident memory held per ciphertext	359,465 B	1,258,455 B
Serialized ciphertext	334,428 B	1,312,467 B
Plaintext vector (5 doubles)	40 B	

Table 6: Memory for the 5-element vector. The first row measures Python objects; the second and third measure the ciphertext itself.

The first row is why we measured memory two ways. On its own, `tracemalloc` suggests a ciphertext costs under a kilobyte, which underestimates it by a factor of about 600: it measures the Python wrapper, not the encrypted data behind it. The resident-set figure and the serialized size are independent measurements of the same object and agree with each other to within 7.5%, which corroborates both.

Five numbers take 40 bytes in the clear and 334 KB encrypted under TenSEAL — an expansion of about 8,361×, and a substantial per-query bandwidth cost, since it is what crosses the network. Section 7 shows the ratio improving sharply once the ciphertext slots are filled.

We also measured the evaluation keys, which the server needs before it can compute at all. At the configuration used in Section 7 they are **179.7 MB** for TenSEAL and **69.2 MB** for OpenFHE, against roughly 5 MB of encrypted data. They are sent once per context rather than per query, but at this configuration they remain the largest object transferred.

5.5 Accuracy

Operation	TenSEAL	OpenFHE
Add	1.57×10^{-9}	4.09×10^{-14}
Multiply	3.35×10^{-6}	1.96×10^{-12}
Sum	4.60×10^{-7}	7.11×10^{-15}
Average	3.10×10^{-7}	5.20×10^{-14}
Dot product	5.19×10^{-6}	3.27×10^{-13}

Table 7: Largest absolute error against the plaintext result.

Both libraries are accurate enough for what CKKS is meant for: an error of 10^{-6} on values of order 10 is sufficiently small for the synthetic analytics workload considered here. Multiplication gives the largest error in both, as the scheme predicts: multiplying two ciphertexts multiplies their noise, and the rescaling afterwards drops low-order bits.

OpenFHE is several orders of magnitude more precise throughout. Its larger scaling factor accounts for part of that, 2^{50} against TenSEAL's 2^{40}, which keeps about ten more bits of every value. Section 6.1 measures the split: matching the parameters removes most of the advantage but not all of it.

6 The Libraries, or the Parameters?

Section 5.1 found TenSEAL faster on every operation and Section 5.5 found OpenFHE more precise. Neither comparison was made at equivalent parameters. We supplied TenSEAL's modulus chain directly, as its API requires, while OpenFHE derived ring dimension 16384 from the depth and 128-bit security level we asked it for, against TenSEAL's 8192. OpenFHE was therefore doing roughly twice the polynomial work per operation and carrying a 2^{50} scaling factor against 2^{40}, keeping ten more bits of every value. This experiment separates the contribution of those parameters from the contribution of the library itself.

OpenFHE lets the ring dimension be set explicitly, so we re-ran it pinned to 8192 with a 2^{40} scale, matching TenSEAL. We also varied OpenFHE's *scaling technique*, an internal option controlling how it manages the scaling factor between multiplications, which TenSEAL does not expose at all.

OpenFHE configuration	Encr	Add	Mult	Sum	Dot	Decr	Geo. mean
OpenFHE initial configuration	2.52	4.02	6.89	7.24	8.70	16.28	**6.44**
Matched, `FLEXIBLEAUTOEXT`	1.20	2.12	3.16	3.25	3.97	7.83	**3.05**
Matched, `FIXEDMANUAL`	1.02	1.77	1.93	2.41	3.09	6.80	**2.37**

Table 8: OpenFHE runtime as a multiple of TenSEAL's, per operation. Lower is closer to TenSEAL.

Matching the parameters closes most of the gap. Multiplication drops from 6.89× to 3.16×, encryption becomes almost identical at 1.20×, and across all six operations the geometric mean falls from 6.44× to 3.05×. Matching the ring dimension and scaling factor together accounts for a factor of 2.18 for multiplication, and changing the scaling technique for a further 1.64.

Two qualifications go with this result. Decryption is still 7.8× slower even matched, so the gap narrows everywhere but closes nowhere. And pinning the ring dimension is not by itself enough. Asked for a ring of 8192 at these parameters, OpenFHE raises an error rather than accepting them, because 8192 does not comply with the 128-bit standard for this modulus [5]; the matched arm runs only because we also set the security level to `HEStd_NotSet`, the setting OpenFHE documents for prototyping. The matched arm therefore holds ring dimension and scale fixed at the price of the 128-bit guarantee — deliberately, because its purpose is to isolate one variable, not to model a deployable configuration. It reports what the libraries do at *equal* parameters, not at equally *secure* ones; the latter comparison is one the 8192 arm cannot make.

6.1 Accuracy at matched parameters

The same three arms split OpenFHE's precision advantage between the scaling factor and the library.

Configuration	Ring / scale	Largest error
TenSEAL	8192 / 2^{40}	3.34×10^{-6}
OpenFHE, initial configuration	16384 / 2^{50}	4.71×10^{-13}
OpenFHE, matched, `FLEXIBLEAUTOEXT`	8192 / 2^{40}	1.77×10^{-9}
OpenFHE, matched, `FIXEDMANUAL`	8192 / 2^{40}	1.05×10^{-8}

Table 9: Largest absolute error on ciphertext–ciphertext multiplication for each arm of the matched-parameter experiment.

Matching the parameters costs OpenFHE most of its precision: the error grows by a factor of 3,757, from 4.71×10^{-13} to 1.77×10^{-9}. Most of the precision gap in Section 5.5 was therefore the scaling factor and the ring, not the library. But the gap does not close. At an identical ring dimension and scaling factor, OpenFHE's absolute error remained approximately 1,889 times smaller than TenSEAL's in this multiplication experiment, so a substantial difference remains that ring dimension and scale alone do not explain. What we take from this is that **a benchmark of two HE libraries at unmatched parameters measures the parameters as much as the libraries**. This may also help explain why Faneela et al. [4] observed a different runtime ordering from the one measured in our initial configurations: unless the configurations are matched, the two are not doing equivalent work.

7 Use Case: Privacy-Preserving Healthcare Analytics

A real workload combines operations, and raises problems that single operations do not. We implemented a complete end-to-end scenario: a hospital wants a cloud provider to compute a score for every patient and the average over the cohort, without exposing any individual record.

The score. We define a **synthetic risk score** over five measurements per patient. Each feature is first normalised to roughly $[0, 1]$ against a fixed public range, $x_{\text{norm}} = (x - \min)/(\max - \min)$, using age 18–90, systolic blood pressure 90–180, BMI 18–40, cholesterol 120–280 and glucose 70–200. The score is then

$$\begin{aligned}\text{score} = 100 \cdot \big(& 0.15\,a + 0.20\,b + 0.15\,m + 0.15\,c + 0.15\,g \\ & + 0.10\,b\,g + 0.05\,m\,c + 0.05\,b^2 \big),\end{aligned}$$

where a, b, m, c, g are the normalised age, blood pressure, BMI, cholesterol and glucose. The eight weights sum to 1, so a patient at the top of every range scores 100.

The shape of the score is what makes it a demanding test. The first five terms are a weighted average and need only multiplication by public constants. The last three are two interaction terms and one square, each multiplying one ciphertext by another. That is the operation that consumes multiplicative depth, and it is why this computation calls for a deeper parameter set than Section 5.

The data. A synthetic cohort of 1000 patients, generated from a fixed seed. Age is drawn uniformly across its range; the four clinical measures are drawn from normal distributions centred on plausible values and then clipped to the ranges above, which keeps every normalised value inside $[0, 1]$. The resulting scores have mean 35.63 and standard deviation 8.45.

Encrypting it. We encrypt one *feature* per ciphertext, one patient per slot, giving five ciphertexts of 1000 values each. A single homomorphic multiplication then advances all 1000 patients at once.

The alternative — one ciphertext per patient — would multiply the number of operations by a thousand.

The padding rule from Section 3 determines the correctness of the cohort mean. Vectors are padded from 1000 to 1024 slots, and we fill the padding with *each feature's minimum*, not with zero. A minimum normalises to exactly 0, so padded slots contribute nothing and drop out of the cohort sum. Zero-padding would instead give those slots a normalised value of $-\min/(\max-\min)$, which is not zero and would shift the average. The padding value is therefore part of the specification of the computation, and a unit test enforces it.

Depth. Normalisation, the interaction terms, the weights and the cohort mean use four levels in total. Neither library can run that at the Section 5 settings: TenSEAL fails with `scale out of bounds`, and a longer modulus chain does not fit at ring 8192, so the ring has to double to 16384. OpenFHE evaluates the score at depth 3 but fails on the cohort mean and needs depth 4. Adding two interaction terms and a square to a weighted average was enough to force new cryptographic parameters and double the ring dimension — in CKKS, the shape of a computation matters more than the amount of data in it.

7.1 Results

Stage	Runs at	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)
Encrypt 5 features	Hospital	—	76.784	84.650
Evaluate the score	Cloud	0.0593	89.249	163.181
Cohort mean	Cloud	0.0748	46.104	146.054
Decrypt the scores	Hospital	—	2.186	17.457
Total		**0.1341**	**214.323**	**411.341**
Overhead		**1×**	**1,598×**	**3,067×**
Per patient		**0.000134**	**0.2143**	**0.4113**

Table 10: The full scenario over 1000 patients. Encryption and decryption have no plaintext counterpart. **The plaintext total is the sum of the separately timed score-evaluation and cohort-mean stages, not a single end-to-end measurement:** we use the sum because our direct end-to-end timing was inconsistent with its own component stages. The overhead ratios therefore inherit the uncertainty of that construction.

	TenSEAL	OpenFHE
Largest absolute error (score points)	4.881×10^{-4}	1.976×10^{-10}
Cohort mean, encrypted	35.627391	35.626948
Cohort mean, plaintext	35.626948	

Table 11: Accuracy of the encrypted score. The worst per-patient error is 5×10^{-4} points on a 0–100 scale.

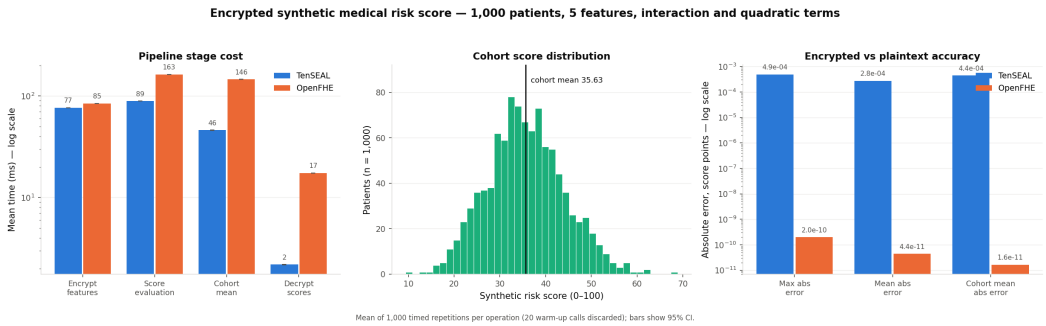

Figure 2: Cost per stage, the distribution of scores across the synthetic cohort, and the accuracy of the encrypted result.

Of the 214 ms (TenSEAL) and 411 ms (OpenFHE) the cloud does 135 ms and 309 ms, and the encrypted cohort mean matches the plaintext value to five significant figures. Against Section 5 that is 1,598× here versus 13,500× for the isolated operations. No operation got faster — the ciphertexts got fuller, 1024 slots carrying real data instead of 5, and a CKKS operation costs what the ring dimension says it costs however many slots hold data. Measuring HE on small, sparsely filled inputs therefore overstates its cost.

The same effect shows up in data volume. Each encrypted feature is about 1 MB, so the hospital uploads roughly 5 MB for a cohort that takes 40 KB in the clear — an expansion of 132×, against the 8,361× measured on the 5-element vector. The evaluation keys, at 179.7 MB, are still larger than the data.

8 Ease of Implementation and Usability

Aspect	TenSEAL	OpenFHE
Installation	`pip install`, any platform	Prebuilt wheel on Linux; source build elsewhere
Writing a computation	Python operators on encrypted vectors	Explicit method calls
Key management	Handled by the context	Evaluation keys generated by hand
Parameters	Modulus chain given directly	Depth given, ring dimension derived
Vector length	Any	Powers of two only
Error messages	Python exceptions	C++ errors through the bindings
Benchmark code	92 lines	111 lines

In our implementation TenSEAL required less code and fewer explicit steps: 92 lines against 111, encrypted expressions that read like the plaintext ones beside them, and key management handled by the context. OpenFHE required more of the programmer — evaluation keys generated explicitly, batch sizes rounded to powers of two, and several parameters that TenSEAL keeps internal. This reflects our experience on this workload rather than a general usability measurement.

That verbosity is not only a cost. OpenFHE requires the multiplicative depth up front, so exceeding it fails immediately and says so, whereas TenSEAL reports `scale out of bounds`, which names the symptom rather than the modulus chain that caused it, so the same failure took us longer to locate. OpenFHE also exposes controls, such as the scaling technique in Section 6, that TenSEAL does not.

9 Practical Limitations

Constraints the libraries impose.

1. **OpenFHE's Python bindings are distributed only for Linux.** The C++ library itself targets Linux, macOS and Windows; it is the prebuilt Python wheel that is published for Ubuntu LTS releases only, so on other platforms the bindings have to be built from source, where TenSEAL ships wheels for all three. We used the wheel, which fixed the platform for the whole project.

2. **Power-of-two batch sizes.** Every input has to be padded, and Section 7 shows the padding value is not always free to choose.
3. **Multiplicative depth has to be planned in advance.** Adding two interaction terms to a weighted average forced new parameters and doubled the ring dimension. Over-requesting is equally costly: depth 5 runs correctly but pushes the ring to 32768 and slows every operation, so the budget has to be matched to the computation in both directions.
4. **Ciphertext memory is invisible to Python.** `tracemalloc` reports a figure about 600 times too small, because the coefficients live in native memory, which is why we measure the resident set as well.

Limitations of this study.

1. Measurements come from one machine, one CPU generation and a single thread. The ratios between columns transfer better than the absolute times.
2. The plaintext baseline for the primitive operations of Section 5 is ordinary Python. A NumPy or C baseline would be faster, which shrinks the denominator of every ratio, so those overhead figures are *lower* bounds relative to optimised plaintext: against a faster baseline the measured overhead would be larger, not smaller. This does not apply to Section 7, whose plaintext risk score is already vectorised with NumPy.
3. Neither library was tested with bootstrapping, so all results concern computations that fit inside a fixed level budget.
4. The benchmark runs client and server in a single process. It therefore measures the computation faithfully, but excludes the network transfer and key management a real deployment would also need.
5. In the healthcare scenario the client performs 79 ms of the 214 ms itself, so at this cohort size outsourcing would not pay for itself on runtime alone. The scenario stands in for larger workloads, where the client-side share is amortised over more records.

10 Conclusions

1. **The overhead is a range, not a number.** Encrypted operations cost between 153× and 33,000× their plaintext equivalents in TenSEAL, and between 615× and 238,000× in OpenFHE. Any single quoted figure hides a factor of two hundred.
2. **What an operation costs depends on whether it needs rotations.** Addition works on ciphertexts directly and is cheap. Summation, average and dot product have to move data between slots, and every rotation carries a key switch. Across the operations we measured, avoiding aggregation across slots was the largest single lever on cost — worth up to two orders of magnitude — though we did not benchmark multiplication by a public constant separately, so we cannot rank it against the ciphertext–ciphertext case.
3. **Much of the difference between the libraries was a difference in parameters.** Re-running OpenFHE at TenSEAL's ring dimension and scale cut its multiplication penalty from 6.89× to 3.16× and made encryption nearly identical. Comparing libraries under unmatched configurations measures configuration differences as well as implementation differences.
4. **Real computations are cheaper than isolated operations suggest, so ciphertexts should be filled.** The healthcare scenario — 1000 records, interaction terms, a cohort aggregate — ran at 1,598× plaintext, an eighth of the isolated benchmark's overhead, and agreed with the plaintext result to five significant figures. No operation became faster; the ciphertexts carried 1024 values instead of 5, and CKKS charges per ciphertext rather than per value. In this configuration the evaluation keys, at 179.7 MB against roughly 5 MB of encrypted data, were the largest data cost.

Homomorphic encryption is usable today for the kind of computation examined here, as long as the workload is batched, structured to avoid unnecessary rotations, and given parameters chosen deliberately rather than by default. The cost is strongly influenced by batching, computation structure and parameter choice, subject to the security level the deployment requires.

Every experiment in this report is provided as a reproducible notebook that installs the libraries and runs the measurement end to end.

References

[1] J. H. Cheon, A. Kim, M. Kim, Y. Song. Homomorphic encryption for arithmetic of approximate numbers. *ASIACRYPT*, 2017.

[2] A. Al Badawi et al. OpenFHE: open-source fully homomorphic encryption library. *WAHC*, 2022.

[3] A. Benaissa, B. Retiat, B. Cebere, A. E. Belfedhal. TenSEAL: a library for encrypted tensor operations using homomorphic encryption. 2021.

[4] Faneela, J. Ahmad, B. Ghaleb, S. U. Jan, W. J. Buchanan. Cross-platform benchmarking of the FHE libraries: novel insights into SEAL and OpenFHE. arXiv:2503.11216, 2025.

[5] M. Albrecht et al. Homomorphic Encryption Security Standard. HomomorphicEncryption.org, 2018.